```
    spec:
      containers:
        - name: catalog-service
          image: catalog-service
          ...
          livenessProbe:                          ❶
            httpGet:                              ❷
              path: /actuator/health/liveness     ❸
              port: 9001                          ❹
            initialDelaySeconds: 10               ❺
            periodSeconds: 5                      ❻
          readinessProbe:                         ❼
            httpGet:
              path: /actuator/health/readiness
              port: 9001
            initialDelaySeconds: 5
            periodSeconds: 15
```

❶ Configuration for the liveness probe

❷ Uses an HTTP GET request to get the liveness state

❸ The endpoint to call for the liveness state

❹ The port to use to fetch the liveness state

❺ An initial delay before starting checking the liveness state

❻ The frequency for checking the liveness state

❼ Configuration for the readiness probe

Both probes can be configured so that Kubernetes will start using them after an initial delay (`initialDelaySeconds`), and you can also define the frequency with which to invoke them (`periodSeconds`). The initial delay should consider that the application will take a few seconds to start, and it will depend on the available computational resources. The polling period should not be too long, to reduce the time between the application instance entering a faulty state and the platform taking action to self-heal.

WARNING If you run these examples on resource-constrained environments, you might need to adjust the initial delay and the polling frequency to allow the application more time to start and get ready to accept requests. You might need to do the same when running these examples on Apple Silicon computers until ARM64 support is part of Paketo Buildpacks (you can follow the updates here: https://github.com/paketo-buildpacks/stacks/issues/51). That's because AMD64 container